

成绩	
评阅人	
日期	2024 年 月 号

重庆邮电大学

学生实验实习报告册

学年学期： 2023-2024 学年 ☐ 春 ☒ 秋学期

课程名称： 人工智能与大数据课程设计

学生学院： 通信与信息工程学院

专业班级： 01012205

学生学号： 2022210410

学生姓名： 雷宇航

联系电话： 18580086873

重庆邮电大学教务处制

课程名称	人工智能与大数据课程设计	课程编号	A2012050
实验地点	YF317	实验时间	2024.12.06
指导教师	胡林		
实验名称	Python 开发环境&语法基础		
评阅人签字		成绩	
一、实验目的 1、熟悉 Python 集成开发系统背景。 2、熟悉 Jupyter Notebook 开发环境。 3、熟悉编写程序的基本过程。 二、实验原理 运用 Anaconda 搭建的 Jupyter Notebook 平台编写实例 Python 程序。 三、使用软件平台 1、Windows 11 电脑一台。 2、Python、Anaconda、Jupyter Notebook 平台。 四、实验步骤、代码、结果及结果分析（复制代码，结果截图） 1. 实例 1：简历卡片 代码： <pre>1. ## 实例一 2. MyInfo = [['姓名', '雷宇航'], ['班级', '01012205'], ['学号', '2022210410'], ['年龄', 20], 3. ['Tel', '18580086873'], ['QQ', '1979613025'], ['E-mail', '1979613025@qq.com']] 4. Characters = ['爱好: 女', '性别: 男', '喜欢: 唱、跳、rap、篮球'] 5. 6. # 个人简历标题 7. print("-" * 16+ '\033[35m 个人简历\033[0m' + "-" * 16) 8. # 基本信息 9. print('\033[36m 基本信息\033[0m') 10.print("-" * 40) 11.for Info in MyInfo: 12. print('{:<10}\t{:<10}'.format(Info[0], str(Info[1]))) 13.print("-" * 40)</pre>			

```

14. # 个人特点
15. print('\033[36m 个人特点\033[0m')
16. print("-" * 40)
17. for char in Characters:
18.     print("    • " + char)
19. print("-" * 40)
20. # 工作经历
21. print('\033[36m 工作经历\033[0m')
22. print("-" * 40)
23. print("曾担任学习委员、现担任辅导员助理、团支书")
24. print("-" * 40)

```

输出：

```

-----个人简历-----
基本信息
-----
姓名          雷宇航
班级          01012205
学号          2022210410
年龄          20
Tel           18580086873
QQ            1979613025
E-mail        1979613025@qq.com
-----
个人特点
-----
    • 爱好：女
    • 性别：男
    • 喜欢：唱、跳、rap、篮球
-----
工作经历
-----
曾担任学习委员、现担任辅导员助理、团支书
-----

```

图 1 简历卡片

2. 实例 2：四则运算

代码：

```

1. ## 实例二
2. print("请输入第一个数：",end="")
3. a = float(input())
4. print("请输入第二个数：",end="")
5. b = float(input())
6. print("\nr%.2f 和%.2f 的和为： %.2f"%(a,b,(a+b)))
7. print("%d 和%d 的差为： %d"%(a,b,(a-b)))
8. print('{}'.format(int(a))+'和{}'.format(int(b))+'的积为：

```

```

    {}'.format(int(a*b)))
9. c = a/b
10. print(F"{int(a)}和{int(b)}的商为: {c:.1f}") #嵌入表达式到字符串常量

```

输出:

```

请输入第一个数: 5
请输入第二个数: 3
5.00和3.00的和为: 8.00
5和3的差为: 2
5和3的积为: 15
5和3的商为: 1.7

```

图 2 四则运算

3. 实例 3: 字符串切片

代码:

```

1. ## 实例三
2. str1 = "*重庆邮电大学-通信与信息工程学院-通信工程专业*"
3. print(" (1) 字符串长度: ",end="")
4. print(len(str1))
5. print(" (2) 切片大学: ",end="")
6. print(str1[1:7])
7. print(" (3) 两次切片拼接: ",end="")
8. print(str1[1:4:2]+str1[8:10])
9. print(" (4) 反向索引: ",end="")
10. print(str1[-7:-6]+str1[-6:-5])
11. print(" (5) 切三个子串并逆序输出第二个字串: ",end="")
12. str2 = str1.split("-",2) #先切割
13. str2 = str2[1] #提取
14. str2=str2[::-1] #逆序
15. print(str2)
16. print(" (6) 通信出现的次数: ",end="")
17. print(str1.count("通信"))
18. print(" (7) 替换通信与信息工程: ",end="")
19. print(str1.replace("通信与信息工程","通信"))

```

输出:

- (1) 字符串长度: 25
- (2) 切片大学: 重庆邮电大学
- (3) 两次切片拼接: 重邮通信
- (4) 反向索引: 通信
- (5) 切三个子串并逆序输出第二个字串: 院学程工息信与信通
- (6) 通信出现的次数: 2
- (7) 替换通信与信息工程: *重庆邮电大学-通信学院-通信工程专业*

图 3 字符串切片

4. 实例 4：计算三角形面积

代码：

```
1. ## 实例四
2. import math
3. a = float(input('a = '))
4. b = float(input('b = '))
5. c = float(input('c = '))
6. if((a+b>c)and(a+c>b)and(b+c>a)):
7.     p = 0.5 * (a + b + c)
8.     s = math.sqrt(p * (p - a) * (p - c) * (p - b))
9.     print("三角形的面积为: %.2f" % s)
10. else:
11.     print("不符合三角形“任意两边之和大于第三边”的条件")
```

输出：

```
a = 1
b = 1
c = 3
不符合三角形“任意两边之和大于第三边”的条件
```

图 4 不符合条件三角形

```
a = 1
b = -1
c = 5
不符合三角形“任意两边之和大于第三边”的条件
```

图 5 不符合条件三角形

```
a = 3
b = 4
c = 5
三角形的面积为: 6.00
```

图 6 符合条件三角形面积结果

5. 实例 5：列表操作

代码：

```
1. ## 实例五
2. from itertools import chain
3. from pypinyin import pinyin, Style #导入拼音库 用于对中文字符排序
4. def to_pinyin(s):
5.     return ''.join(chain.from_iterable(pinyin(s, style=Style.TONE3))
6. )
7. city = input("请输入 5 个城市的名字，用空格分隔：")
8. City = city.split(" ",4)
9. print('for 循环遍历输出：')#利用 for 循环输出城市名
```

```
9. for i in City:
10.     print('{}'.format(i),end=" ")
11. print('\n 索引值输出: {}'.format(City[0]))
12. print('切片输出: ',City[1:5])
13. print('逆序输出: ',end="")#各种方法实现逆序输出
14. print('切片逆序: ',end="")
15. print(City[::-1])
16. cit2 = City.copy() # reverse 是就地操作 会改变原列表 所以需要复制一份
17. cit2.reverse()
18. print('reverse 逆序: ',end="")
19. print(cit2)
20. cit3 = City.copy()
21. print('sort: ',end="")#两种方法实现排序 sort 就地 sorted 是新列表
22. cit3.sort(reverse=True,key=to_pinyin) #利用 reverse 实现降序
23. print(cit3)
24. cit4 = City.copy()
25. print('sorted: ',end="")
26. print(sorted(cit4,reverse=True,key=to_pinyin))
27. a = cit3.index('重庆')#利用 index 函数查询下标
28. print("我的家乡城市: ",cit3[a])
29. l = len(cit3) #城市列表长度
30. #输出家乡前后的城市
31. if (a-1) >= 0:
32.     print('前面的城市是: ',cit3[0:a])
33. if (a+1) <= (l-1):
34.     print('后面的城市是: ',cit3[a+1:l])
35. #修改前后城市名
36. if (a-1) >= 0:
37.     cit3[a-1] = input("请输入城市名: ")
38.     print("前面修改完成!")
39. else:
40.     print('前面无其他城市')
41. if (a+1) <= (l-1):
42.     cit3[a+1] = input("请输入城市名: ")
43.     print("后面修改完成")
44. else:
45.     print('后面无其他城市')
46. print(cit3)
47. #删除城市并输出
48. if (a-1) >= 0:
49.     cit3.pop(a-1)
50.     if(a+1)<=len(cit3): # pop 默认弹出一个 总长度减一
51.         cit3.pop(a)
52. else:
53.     if (a+1) <= (l-1):
54.         cit3.pop(a+1)
```

```
55.print("剩余的城市为: ",cit3,end="")
```

输出:

```
请输入5个城市的名字,用空格分隔: 重庆 上海 武汉 南京 北京
for循环遍历输出:
重庆 上海 武汉 南京 北京
索引值输出: 重庆
切片输出: ['上海', '武汉', '南京', '北京']
逆序输出: 切片逆序: ['北京', '南京', '武汉', '上海', '重庆']
reverse逆序: ['北京', '南京', '武汉', '上海', '重庆']
sort: ['武汉', '上海', '南京', '重庆', '北京']
sorted: ['武汉', '上海', '南京', '重庆', '北京']
我的家乡城市: 重庆
前面的城市是: ['武汉', '上海', '南京']
后面的城市是: ['北京']
请输入城市名: 广东
前面修改完成!
请输入城市名: 成都
后面修改完成
['武汉', '上海', '广东', '重庆', '成都']
剩余的城市为: ['武汉', '上海', '重庆']
```

图 7 列表操作

6. 实例 6: 字典操作

代码:

```
1. ## 实例六
2. dic_country={"China":"Beijing","America":"Washington","Norway":"Oslo",
  "Japan":"Tokyo","Germany":"Berlin","Canada":"Ottawa","France":"Paris",
  "Thailand":"Bangkok"}
3. country_name=input("请输入国家名:").lower().capitalize() #先小写 再首字母大写
4. if country_name in dic_country:
5.     print("首都名:{}".format(dic_country[country_name]))
6. else:
7.     print("未查询到该国家名!")
```

输出:

```
请输入国家名: SGD
未查询到该国家名!
```

图 8 无法查询国家

```
请输入国家名: China
首都名:Beijing
```

图 9 可正常查询国家

7. 思考题 1: 列表嵌套

代码:

```
1. ## 思考题一
2. import itertools
3. nested_list_510 = [[6,7,5],[8,10,9],[11,13,12]]
4. answer = [sorted(map(int, i)) for i in nested_list_510]#对嵌套列表中的
   每一行数据进行排序
5. print(answer)
6. list_510 = list(itertools.chain.from_iterable(answer))#对列表进行压平
7. for i in range(0,9):
8.     print(list_510[i],end=" ")
```

输出:

```
[[5, 6, 7], [8, 9, 10], [11, 12, 13]]
5 6 7 8 9 10 11 12 13
```

图 10 列表嵌套及压平

8. 思考题 2: 成绩排序

代码:

```
1. dic_score = {
2.     "张三": [88, 90, 98, 95],
3.     "李四": [85, 92, 95, 98],
4.     "王五": [89, 89, 90, 92],
5.     "丁六": [82, 86, 89, 90]
6. }
7.
8. # (1) 统计每名学生的平均成绩和最高成绩, 添加至每个学生成绩的后面, 最后将字典
   输出。
9. for name, scores in dic_score.items():
10.     average_score = sum(scores) / len(scores)
11.     max_score = max(scores)
12.     dic_score[name].extend([average_score, max_score])
13.
14. print(" (1) 修改后学生成绩: ")
15. print(dic_score)
16.
17. # (2) 将 (1) 中的成绩形成嵌套列表输出。
18. nested_list = [[name] + scores for name, scores in dic_score.items()]
19. print(" (2) 嵌套列表学生成绩: ")
20. print(nested_list)
21.
22. # (3) 求每门课的最高分, 作为新的键和值存入字典, 并按照如下格式输出。
23. MaxScore=[0,0,0,0,0,0]
24. for scores in dic_score.values():
25.     MaxScore[0] = max(MaxScore[0], scores[0])
```



```

26.     MaxScore[1] = max(MaxScore[1], scores[1])
27.     MaxScore[2] = max(MaxScore[2], scores[2])
28.     MaxScore[3] = max(MaxScore[3], scores[3])
29.     MaxScore[4] = max(MaxScore[4], scores[4])
30.     MaxScore[5] = max(MaxScore[5], scores[5])
31. dic_score["最高分"]=MaxScore
32. print("（3）最终成绩表：")
33. print("姓名\t语文\t数学\t英语\t计算机\t平均分\t最高分")
34. for name, scores in dic_score.items():
35.     print(f"{name}\t{scores[0]}\t{scores[1]}\t{scores[2]}\t{scores[3]}\t{scores[4]:.2f}\t{scores[5]}")

```

输出：

（1）修改后学生成绩：
{'张三': [88, 90, 98, 95, 92.75, 98], '李四': [85, 92, 95, 98, 92.5, 98], '王五': [89, 89, 90, 92, 90.0, 92], '丁六': [82, 86, 89, 90, 86.75, 90]}

（2）嵌套列表学生成绩：
[['张三', 88, 90, 98, 95, 92.75, 98], ['李四', 85, 92, 95, 98, 92.5, 98], ['王五', 89, 89, 90, 92, 90.0, 92], ['丁六', 82, 86, 89, 90, 86.75, 90]]

（3）最终成绩表：

姓名	语文	数学	英语	计算机	平均分	最高分
张三	88	90	98	95	92.75	98
李四	85	92	95	98	92.50	98
王五	89	89	90	92	90.00	92
丁六	82	86	89	90	86.75	90
最高分	89	92	98	98	92.75	98

图 11 列表嵌套及压平

五、碰到的问题及解决办法

1. pop()的机制

pop()可以移除并返回列表中的某个元素，还允许通过索引来指定需要移除的元素，若不提供索引，pop()会默认移除并返回列表中的最后一个元素。在实验过程之初，没有注意到 pop()调用过后列表长度会减少 1，导致在实例 5 中对城市进行连续前后 pop()的时候出现报错。因此根据我所在城市是否为第一个城市将前后城市删除写进了一个 if、else 语句，若删除前面的城市，则需要通过调整索引的范围，实现后面城市的删除。

六、实验心得体会

在本次 Python 实验过程中，我不仅巩固了对 Python 基础语法的理解和应用，更重要的是，我在原有基础上取得了显著的进步，学习到了许多新知识。例如，我深入掌握了列表的多种操作，包括切片、排序等高级技巧，并且学会了如何利用导入模块的方法来解决实际问题。总的来说，Python 是一门极易入门的编程语言，其内置丰富的函数和模块为初学者提供了极大的便利，使我深

刻感受到了 Python 这门语言的独特魅力，极大地激发了我的学习兴趣。通过进一步了解，我发现 Python 还能实现诸多强大功能，如数据挖掘、网络爬虫和绘图等。我热切期望通过不断学习，能够熟练运用这些功能，将 Python 的潜力充分发挥出来！